Lecture 6: POS Tagging & Sentiment Analysis

Week 2: From Structure to Meaning

LLM Course

Learning Objectives

By the end of this lecture, you will:

1. Understand part-of-speech (POS) tagging and its applications
2. Explore how neural networks learn grammatical structure
3. Apply sentiment analysis to real-world text
4. Fine-tune pre-trained models for domain-specific tasks
5. Critically evaluate whether models ”understand” language

Central questions:

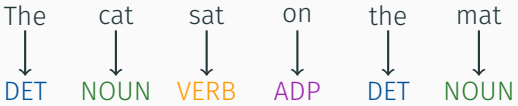
- Can statistical patterns capture grammatical knowledge?
- What does it mean for a model to ”understand” emotion?

Part-of-Speech (POS) Tagging

What is POS tagging?

- Assigning grammatical category to each word
- Categories: noun, verb, adjective, adverb, pronoun, preposition, etc.
- A fundamental NLP task

Example:



Why it matters:

- Disambiguation: ”book” as noun vs. verb
- Syntax parsing and understanding

POS Tagsets: Universal vs. Fine-Grained [?]

Universal POS Tags (17 tags):

- ADJ: adjective
- ADP: adposition
- ADV: adverb
- AUX: auxiliary
- CONJ: conjunction
- DET: determiner
- NOUN: noun
- NUM: numeral
- PRON: pronoun
- PROPN: proper noun
- VERB: verb

Penn Treebank (45+ tags):

- NN: noun, singular
- NNS: noun, plural
- NNP: proper noun, singular
- NNPS: proper noun, plural
- VB: verb, base form
- VBD: verb, past tense
- VBG: verb, gerund
- VBN: verb, past participle
- VBP: verb, non-3rd person
- VBZ: verb, 3rd person
- ...

Context Matters: Ambiguous Words 🤔

Many words have multiple possible POS tags:

Sentence	Word	POS
"I **book** a flight"	book	VERB
"I read a **book**"	book	NOUN
"The **fast** car"	fast	ADJ
"She runs **fast**"	fast	ADV
"I will **fast** today"	fast	VERB
"The meeting is **close**"	close	ADJ
"Please **close** the door"	close	VERB

Key insight: Context determines POS! Models must look at surrounding words.

POS Tagging with spaCy

```python
import spacy

nlp = spacy.load("en_core_web_sm")

sentence = "She will book the meeting room tomorrow"
doc = nlp(sentence)

print(f"{'Word':<12} {'POS':<8} {'Detailed Tag':<12} {'Explanation
    '}")
print("-" * 55)
for token in doc:
    explanation = spacy.explain(token.pos_)
    print(f"{token.text:<12} {token.pos_:<8} {token.tag_:<12} {
        explanation}")

# Output:
```

How Do POS Taggers Work?

Traditional approaches (pre-neural):

- Rule-based: Hand-crafted grammar rules
- Hidden Markov Models (HMMs): Probabilistic sequences
- Conditional Random Fields (CRFs): Structured prediction

Modern neural approaches:

- Recurrent Neural Networks (RNNs/LSTMs): Process sequences
- Transformers (BERT, etc.): Bidirectional context
- Fine-tune pre-trained models on POS data

Advantages of neural methods:

- Learn patterns from data (no hand-crafted rules)
- Capture long-range dependencies
- Handle ambiguity through context

Neural Networks and Grammar [?]

Can neural networks learn syntactic structure?

Classic test: Subject-verb agreement (Linzen et al., 2016)

Must track "key" not "cabinets"

- "The key to the cabinets **is** on the table" [?]
- "The key to the cabinets **are** on the table" [?]

Challenge: Distractor nouns between subject and verb

- Model must identify "key" (singular) as subject
- Ignore "cabinets" (plural distractor)
- Predict correct verb form "is" (not "are")

Finding: LSTMs can learn this! But they struggle with complex cases.

BLiMP: Testing Linguistic Knowledge [?]

BLiMP Dataset (Warstadt et al., 2020):

- Benchmark of Linguistic Minimal Pairs
- 67,000 sentence pairs across 67 paradigms
- Tests syntax, semantics, morphology

Example minimal pairs:

Acceptable	Unacceptable
”The author writes”	”The author write”
”Who did you see?”	”Who did you saw?”
”I think that she left”	”I think that she leave”

Task: Model should assign higher probability to acceptable sentence

Token Classification with HuggingFace 🤗

```python
from transformers import pipeline

# Load POS tagging pipeline (or NER, etc.)
pos_tagger = pipeline("token-classification",
                      model="vblagoje/bert-english-uncased-
                          finetuned-pos",
                      aggregation_strategy="simple")

sentence = "Apple Inc. is looking at buying a UK startup"
results = pos_tagger(sentence)

for result in results:
    print(f"{result['word']:<15} {result['entity_group']:<8} "
          f"(confidence: {result['score']:.3f})")

# Output:
```

Discussion: Do Models "Understand" Grammar? 🤔

Perspectives to consider:

1. **Chomsky's view:** Grammar requires innate, symbolic rules
   - Can statistical patterns truly capture grammatical knowledge?
2. **Emergentist view:** Grammar emerges from usage patterns
   - Maybe neural networks learn similarly to humans?
3. **Functional perspective:** If it works, does it matter?
   - Models perform well on tasks—is that "understanding"?
4. **Limitations:** Models still fail on edge cases
   - What does this tell us about their knowledge?

Your thoughts? Is pattern matching sufficient for grammatical competence?

Sentiment Analysis: From Structure to Emotion

What is sentiment analysis?

- Determining emotional tone of text
- Classification task: positive, negative, neutral (or fine-grained)
- Moves beyond structure to *meaning*

Applications:

- Social media monitoring
- Customer feedback analysis
- Market research and stock prediction
- Movie/product review analysis
- Political opinion tracking

Granularity levels:

Sentiment Analysis Challenges 🤔

1. **Sarcasm and irony:**
   - "Oh great, another meeting 🙄" (negative, despite "great")
   - "This is the best movie I've ever fallen asleep to" (negative!)
2. **Context-dependent sentiment:**
   - "This movie is sick!" (positive in slang, negative literally)
   - "The book was long" (neutral? negative?)
3. **Mixed sentiment:**
   - "Great food but terrible service" (both positive and negative)
   - Aspect-based sentiment: food=positive, service=negative
4. **Negation:**
   - "not good" vs. "good"

Sentiment Lexicons: The Old-School Approach

Traditional method: Dictionary of words with sentiment scores

Popular lexicons:

- **AFINN:** Words rated from -5 (very negative) to +5 (very positive)
- **SentiWordNet:** Assigns positive/negative/neutral scores
- **VADER:** Designed for social media text (handles emoji, slang)

Example (AFINN):

Word	Score
love	+3
great	+3
good	+3
hate	-3

Lexicon-Based Sentiment Analysis

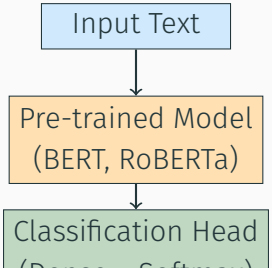
```
from vaderSentiment.vaderSentiment import
   SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

texts = [
    "I love this product! It's amazing!",
    "This is the worst experience ever.",
    "It's okay, nothing special.",
    "Great food but terrible service! "
]

for text in texts:
    scores = analyzer.polarity_scores(text)
    print(f"Text: {text}")
    print(f"  Positive: {scores['pos']:.2f}, "
```

Modern Approach: Pre-trained Models 🤗

Neural network advantages:

- Learn context-dependent representations
- Capture word order and negation
- Handle sarcasm better (though still imperfect)
- Transfer learning: pre-train on large corpus, fine-tune for sentiment

Typical architecture:



Sentiment Analysis with HuggingFace 🤗

```python
from transformers import pipeline

# Load pre-trained sentiment analysis pipeline
sentiment_analyzer = pipeline("sentiment-analysis")

texts = [
    "I love this product! It's amazing!",
    "This is the worst experience ever.",
    "It's okay, nothing special.",
    "I don't hate it, but I don't love it either."
]

for text in texts:
    result = sentiment_analyzer(text)[0]
    print(f"Text: {text}")
    print(f"  Sentiment: {result['label']}, "
```

Domain-Specific Sentiment Models

Problem: General models may not understand domain-specific language

Solution: Fine-tune on domain-specific data!

Domain	Model	Example Use
Financial	FinBERT	Earnings sentiment
Medical	BioBERT + fine-tune	Patient feedback
Twitter	TwitterBERT	Social monitoring
Product Reviews	RoBERTa + Amazon	E-commerce
Movies	BERT + IMDb	Film reviews

Why it works:

- Domain-specific vocabulary (”bullish” in finance = positive)
- Different sentiment expressions

Comparing General vs. Domain-Specific

```python
from transformers import pipeline

# General sentiment model
general = pipeline("sentiment-analysis")

# Financial sentiment model
financial = pipeline("sentiment-analysis",
                     model="ProsusAI/finbert")

texts = [
    "The company's earnings exceeded expectations",
    "Revenue declined but margins improved",
    "Stock prices plummeted amid uncertainty"
]

for text in texts:
```

Fine-Tuning for Sentiment Analysis

Process overview:

1. **Start with pre-trained model** (e.g., BERT, RoBERTa)
   - Already knows language structure from pre-training
2. **Prepare labeled dataset**
   - Text + sentiment labels (positive/negative/neutral)
   - Examples: IMDb reviews, Amazon products, Twitter data
3. **Add classification head**
   - Dense layer on top of model
   - Outputs: probability distribution over classes
4. **Fine-tune on sentiment data**
   - Update model weights to predict sentiment
   - Much faster than training from scratch!
5. **Evaluate on test set**
   - Accuracy, precision, recall, F1-score

 HuggingFace Chapter 3.2: Processing Data for Fine-tuning

Fine-Tuning Example (Simplified)

```
from transformers import AutoModelForSequenceClassification,
    Trainer

# 1. Load pre-trained model
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2  # positive/negative
)

# 2. Prepare data (tokenized with labels)
# train_dataset, eval_dataset = ... (omitted for brevity)

# 3. Define training arguments
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
```

Evaluation Metrics for Sentiment Analysis

Beyond accuracy:

- **Precision:** Of predicted positives, how many are truly positive?

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall:** Of actual positives, how many did we catch?

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

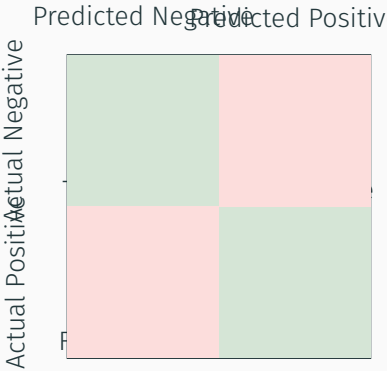
- **F1-Score:** Harmonic mean of precision and recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why not just accuracy?

- Imbalanced datasets (e.g., 90% positive reviews)
- Different costs for false positives vs. false negatives

Confusion Matrix Example



Accuracy = 92% | Precision = 95% | Recall = 90% | F1 = 92%

Aspect-Based Sentiment Analysis [?]

Problem: Reviews often mention multiple aspects with different sentiments

Example:

"The food was delicious but the service was terrible. The atmosphere was okay."

Aspect-based approach:

- Food: Positive [?]
- Service: Negative [?]
- Atmosphere: Neutral [?]

Applications:

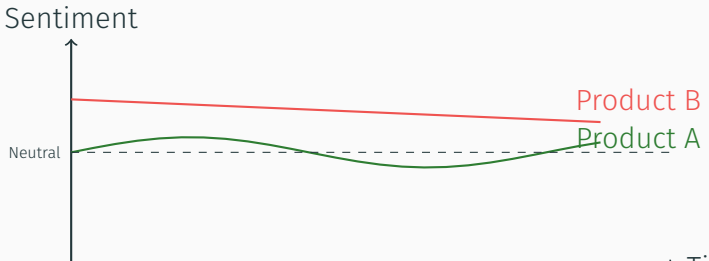
- Restaurant reviews: food, service, ambiance, price
- Product reviews: quality, price, shipping, customer service

Real-World Application: Product Review Analysis

Business value:

- Identify product strengths and weaknesses
- Track sentiment trends over time
- Compare against competitors
- Prioritize product improvements

Example insights from Amazon reviews:



Hands-On Exercise

Try this yourself:

1. **Collect data:**
   - Scrape product reviews (Amazon, Yelp)
   - Or use public dataset (IMDb, Twitter)
2. **Compare approaches:**
   - Lexicon-based (VADER)
   - General pre-trained model (HuggingFace pipeline)
   - Domain-specific model (if available)
3. **Analyze results:**
   - Where do models disagree?
   - Which handles sarcasm better?
   - Which is most accurate for your domain?
4. **Bonus:** Fine-tune a model on your specific dataset!

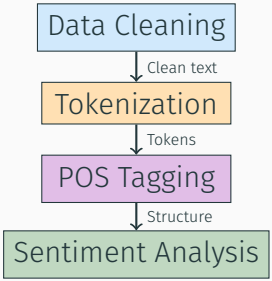
Discussion: Understanding Emotion

Philosophical questions:

1. Can models ”feel” sentiment?
   - They predict labels, but do they understand emotion?
2. Is sentiment objective or subjective?
   - Different annotators may disagree on sentiment
   - How do models handle ambiguity?
3. Cultural and linguistic variation:
   - Sentiment expressions vary across cultures
   - Can models capture these nuances?
4. Ethical considerations:
   - Automated sentiment analysis in hiring, lending...
   - Risks of bias and discrimination
   - Should we trust model judgments?

Week 2 Integration: Putting It All Together

How the pieces connect:



Remove noise, normalize text

Break into units

Identify grammar

Extract meaning

Each step builds on the previous one!

Assignment 2: SPAM Email Classifier

Your task: Build a classifier to detect spam emails

Apply this week's concepts:

- **Data cleaning:** Remove HTML tags, normalize text
- **Tokenization:** Try different tokenizers (word, subword)
- **Features:** Extract useful signals (POS patterns, sentiment?)
- **Classification:** Train a model to identify spam

Think about:

- What makes spam different from legitimate emails?
- How does preprocessing affect accuracy?
- Can you use sentiment as a feature?
- What about POS patterns? (e.g., spam has more imperatives?)

Key Takeaways

1. **POS tagging reveals grammatical structure**
   - Neural networks can learn syntax from data
   - But do they ”understand” grammar? Debatable!
2. **Sentiment analysis extracts emotional meaning**
   - From lexicons to neural networks
   - Domain-specific fine-tuning improves accuracy
3. **Statistical learning powers modern NLP**
   - Models learn patterns without explicit rules
   - Similar to how infants learn language?
4. **Context is crucial**
   - Words are ambiguous—meaning comes from context
   - Transformers excel at capturing context
5. **Critical thinking matters**
   - Question what ”understanding” means
   - Be aware of limitations and biases

Primary References

POS Tagging and Syntax:

- Linzen, Dupoux, & Goldberg (2016). Assessing the Ability of LSTMs to Learn Syntax-Sensitive Dependencies. *TACL*.
- Warstadt et al. (2020). BLiMP: The Benchmark of Linguistic Minimal Pairs. *TACL*.

Sentiment Analysis:

- Pang & Lee (2008). Opinion Mining and Sentiment Analysis. *Foundations and Trends in IR*.
- Hutto & Gilbert (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis. *ICWSM*.

HuggingFace Resources:

- Chapter 1.2: NLP Tasks with Pipeline

Looking Ahead: Weeks 3-4 🔮

Next topics:

- Dimensionality reduction (PCA, UMAP)
- Bag-of-words and TF-IDF
- Word embeddings (Word2Vec, GloVe)
- Distributional semantics

Central question:

"You shall know a word by the company it keeps"

How can we represent word *meaning* computationally?

Prepare by:

- Completing Assignment 2

Additional Resources

Tools and libraries:

- spaCy: `https://spacy.io/`
- HuggingFace Transformers: `https://huggingface.co/docs/transformers/`
- VADER Sentiment: `https://github.com/cjhutto/vaderSentiment`

Datasets:

- IMDb Movie Reviews: `https://ai.stanford.edu/~amaas/data/sentiment/`
- Amazon Product Reviews: `https://registry.opendata.aws/amazon-reviews/`
- Stanford Sentiment Treebank: `https://nlp.stanford.edu/sentiment/`

Thank you!

Questions?

Week 2 complete! See you in Week 3!

Happy coding!